MANUAL PRÁTICO
CRUD READ com JavaScript e Node.js

Tutorial guiado para aprendizagem autônoma | Curso Técnico em Desenvolvimento de Sistemas | Aulas 14 e 15

Objetivo. Aprender a recuperar dados persistidos em arquivos JSON, compreender os resultados e construir consultas sem depender de soluções prontas. Este manual acompanha os dois vídeos e os manuais das atividades.

Você vai precisar: VS Code, Node.js instalado e um terminal. Trabalho individual; registre dúvidas, erros, testes e correções.

Mapa de aprendizagem

• Etapa 1 - Preparar uma pasta e reconhecer o JSON.

• Etapa 2 - Ler o arquivo como texto com fs.readFileSync().

• Etapa 3 - Converter o texto com JSON.parse().

• Etapa 4 - Percorrer registros com forEach().

• Etapa 5 - Localizar o primeiro registro com find().

• Etapa 6 - Filtrar vários registros com filter().

• Etapa 7 - Testar casos sem resultado e analisar erros.

• Etapa 8 - Aplicar os conceitos no projeto da atividade.

Regra de ouro: digite os exemplos, execute-os, compare com a saída prevista e explique o que aconteceu. Copiar sem testar não conclui a etapa.

1. Monte o laboratório de exemplos

Crie uma pasta chamada laboratorio-read e abra-a no VS Code. Dentro dela, crie dois arquivos: maquinas.json e laboratorio.js. Este laboratório tem dados próprios para não resolver diretamente o projeto avaliativo.

Arquivo maquinas.json - digite exatamente:

```
[
  { "id": 1, "nome": "Esteira A", "setor": "Montagem", "status": "Ativa" },
  { "id": 2, "nome": "Prensa B", "setor": "Estamparia", "status": "Manutenção" },
  { "id": 3, "nome": "Torno C", "setor": "Usinagem", "status": "Ativa" },
  { "id": 4, "nome": "Fresa D", "setor": "Usinagem", "status": "Manutenção" }
]
```

Observe: o arquivo contém um array (colchetes) com quatro objetos (chaves). Cada objeto tem propriedades e valores. JSON exige aspas duplas nas propriedades e nos valores textuais.

Conferência rápida

• Quantos objetos existem no array?

• Qual propriedade representa o identificador único?

• Qual é o tipo de dado do valor da propriedade id?

• Quais máquinas estão no setor Usinagem?

2. Leia o arquivo como texto

No arquivo laboratorio.js, digite:

```
const fs = require("fs");

const texto = fs.readFileSync("maquinas.json", "utf8");
console.log(typeof texto);
console.log(texto);
```

No terminal do VS Code, na pasta do projeto, execute:

```
node laboratorio.js
```

Saída esperada: a primeira linha será string; depois aparecerá o texto do JSON, com colchetes, chaves e propriedades.

Entenda: require("fs") carrega o módulo de sistema de arquivos do Node.js. readFileSync(caminho, codificacao) lê o arquivo de modo síncrono. Com "utf8", retorna seu conteúdo em texto. O caminho relativo é interpretado a partir do diretório de trabalho no terminal.

Experimente

Troque temporariamente o nome do arquivo no código por arquivo-inexistente.json e execute. Anote a mensagem de erro. Depois, restaure maquinas.json.

3. Transforme o texto em dados JavaScript

Substitua o código de laboratorio.js por:

```
const fs = require("fs");
const texto = fs.readFileSync("maquinas.json", "utf8");
const maquinas = JSON.parse(texto);

console.log(Array.isArray(maquinas));
console.log(maquinas.length);
console.log(maquinas[0].nome);
```

Saída esperada: true, depois 4, depois Esteira A.

Entenda: JSON.parse() converte um texto JSON válido em um valor JavaScript. Neste caso específico, o valor é um array de objetos. Array.isArray() verifica se o valor é um array; length informa a quantidade de elementos; [0] acessa o primeiro item.

Atenção: JSON.parse() não cria sempre um array; isso depende da estrutura do JSON original.

Experiência guiada

Exiba o nome da última máquina usando um índice numérico. Depois, altere o código para mostrar o setor da segunda máquina. Execute a cada mudança.

4. Listar registros com forEach()

Mantenha a leitura e a conversão do exemplo anterior e substitua os três console.log() por:

```
maquinas.forEach((maquina, indice) => {
  console.log(`${indice + 1}. ${maquina.nome}`);
});
```

Saída esperada: quatro linhas numeradas, de 1. Esteira A a 4. Fresa D.

O método forEach() chama a função informada para cada elemento do array. Aqui, maquina recebe o objeto atual e indice recebe a posição desse objeto. O método não gera um novo array como valor de retorno.

Teste de alteração

Altere a saída para mostrar nome - setor em cada linha. Em seguida, tente incluir também a situação da máquina. Compare sua saída com os quatro objetos do arquivo.

5. Buscar um único registro com find()

Para evitar repetir o código de carregamento a cada exercício, mantenha as três instruções iniciais de leitura e conversão. Em seguida, digite:

```
const procurada = maquinas.find(maquina => maquina.id === 2);
console.log(procurada);
```

Saída esperada: o objeto da Prensa B, ID 2.

O método find() percorre o array e devolve o primeiro elemento que atende à condição. O operador === compara valor e tipo. Se não houver correspondência, o resultado será undefined.

Teste importante: registro inexistente

```
const procurada = maquinas.find(maquina => maquina.id === 99);
console.log(procurada); // undefined
```

Para fazer uma consulta segura, verifique o retorno antes de acessar propriedades:

```
if (procurada !== undefined) {
  console.log(procurada.nome);
} else {
  console.log("Máquina não encontrada.");
}
```

Experimente: faça a busca pelo ID 3 e confirme o nome encontrado. Em seguida, busque o ID 30 e verifique a mensagem. Compare a busca por 2 e por "2" (string).

6. Selecionar vários registros com filter()

Mantenha as instruções de carregamento e digite:

```
const emManutencao = maquinas.filter(
  maquina => maquina.status === "Manutenção"
);

console.log(emManutencao.length);
console.log(emManutencao);
```

Saída esperada: o número 2 e um array com os objetos Prensa B e Fresa D.

Entenda: filter() devolve um novo array com todos os elementos aprovados na condição. Quando nenhum elemento atende à condição, retorna [], que é um array vazio, não undefined.

Teste sem resultados

```
const paradas = maquinas.filter(maquina => maquina.status === "Parada");
console.log(paradas);        // []
console.log(paradas.length); // 0
```

Experimente: filtre as máquinas do setor Usinagem. Depois, altere a condição para setor Montagem. Verifique quantos objetos cada busca retorna.

7. Compare os três métodos

Método	O que faz	Retorno
forEach()	Executa uma função por item	undefined
find()	Retorna o primeiro item que corresponde	Elemento ou undefined
filter()	Seleciona todos os itens que correspondem	Novo array, inclusive []

Questão para reflexão: se a indústria precisa listar todas as máquinas com defeito, find() seria a melhor escolha? Explique por quê.

8. Erros comuns e como investigar

ENOENT: arquivo não encontrado

Confira o nome e a localização do arquivo. Execute o comando no terminal dentro da pasta que contém o JSON; confira também a extensão.

SyntaxError no JSON.parse()

Abra o JSON e confira aspas duplas, vírgulas e chaves. Utilize um formato JSON válido antes de tentar converter.

Cannot read properties of undefined

Uma busca find() pode não ter retornado nada. Verifique o resultado antes de acessar propriedades.

filter() não encontra nada

Confira exatamente o texto de comparação: maiúsculas, minúsculas, acentos e espaços fazem diferença.

Resultado inesperado ao comparar IDs

O número 2 é diferente da string "2" ao utilizar ===. Verifique os tipos com typeof.

readFileSync() trava a execução durante a leitura

É uma operação síncrona. Funciona bem neste laboratório, mas não é a opção ideal para leituras frequentes em servidores com múltiplas requisições.

9. Treino opcional: tratando um erro de leitura

Este exercício demonstrativo ajuda a reconhecer falhas de arquivo. Não substitui a verificação dos retornos de find() e filter().

```
const fs = require("fs");

try {
  const texto = fs.readFileSync("maquinas.json", "utf8");
  const maquinas = JSON.parse(texto);
  console.log(`Registros carregados: ${maquinas.length}`);
} catch (erro) {
  console.log("Não foi possível carregar os dados.");
  console.log(erro.message);
}
```

Execute com o nome certo e, depois, com um nome inexistente. Observe a diferença entre falha na leitura e consulta que retorna zero registros.

10. Desafios progressivos - sem gabarito

Desafio A - Fundamental. Exiba uma linha para cada máquina no padrão ID | Nome | Status. Utilize o array carregado do JSON.

Desafio B - Intermediário. Leia um ID de uma variável, localize a máquina e mostre o setor. Se não existir, apresente uma mensagem amigável.

Desafio C - Intermediário. Crie uma variável com o nome de um setor. Exiba quantas máquinas pertencem a ele e os nomes encontrados.

Desafio D - Avançado. Crie uma função que receba um status por parâmetro e devolva o array filtrado. Teste com status existente e inexistente.

Desafio E - Transferência. Abra o projeto oficial da atividade no Classroom e identifique quais estruturas e métodos deste laboratório são úteis para implementar as funções pedidas. Não copie o laboratório sem adaptar ao formato dos dados reais.

11. Evidências e relatório de aprendizagem

Prepare um documento em editor de texto e exporte para PDF. Ele deve conter: objetivo da atividade; trechos de código que você desenvolveu; imagens legíveis dos testes no terminal; explicação de pelo menos um erro encontrado e como foi resolvido; respostas aos itens de reflexão.

Organização da entrega: envie ao Classroom o JavaScript desenvolvido (ou ZIP do projeto, conforme a atividade) e o relatório em PDF. Não é necessário enviar node_modules.

Checklist de autoverificação

- ☐ Consigo explicar a diferença entre o texto JSON e o array JavaScript.
- ☐ Consigo executar node laboratorio.js na pasta correta.
- ☐ Sei identificar o retorno de forEach(), find() e filter().
- ☐ Testei pelo menos uma consulta que encontra resultado e outra que não encontra.
- ☐ Conferi os nomes e tipos das propriedades utilizadas nas comparações.
- ☐ Registrei testes, erros e correções antes de entregar.

Mensagem final. Use o NotebookLM para esclarecer dúvidas e investigar conceitos, mas escreva, execute e explique seu próprio código. A compreensão é demonstrada quando você consegue adaptar a solução a um conjunto de dados diferente.